

Scraped Products review data sentimental analysis using Machine Learning

A CAPSTONE PROJECT REPORT

*Submitted in partial fulfillment of the
requirement for the award of the
Degree of*

**BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING**

by

**Sikhakolli Phanindra (19BCI7012)
Desaboina Deekshitha(19BCI7068)
Mamidala Taran-(19BCE7346)**

Under the Guidance of

DR. Sandipan Maiti



**SCHOOL OF COMPUTERS ENGINEERING
VIT-AP UNIVERSITY
AMARAVATI- 522237**

JANUARY 2023

CERTIFICATE

This is to certify that the Capstone Project work titled “**Scraped Products review data sentimental analysis using Machine Learning**” that is being submitted by **Sikhakolli Phanindra (19BCI7012), Desaboina Deekshitha(19BCI7068), Mamidala Taran (19BCE7346)** is in partial fulfilment of the requirements for the award of Bachelor of Technology, is a record of bonafide work done under my guidance. The contents of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for award of any degree or diploma and the same is certified.

Dr. Sandipan Maiti
Guide

The thesis is satisfactory / unsatisfactory

Internal Examiner
Examiner

External

Approved By

PROGRAM CHAIR
DEAN

B.Tech. SE
School of Computer Science and Engineering

ACKNOWLEDGEMENTS

We the students of the Computer Science and Engineering department, Bachelor of Technology 7th Semester, do hereby acknowledge our gratefulness towards all the people associated in the completion of this project. we would like to pay our thanks to our respected project mentor Dr. Sandipan Maiti who has chosen us for this project and also provided us help with knowledge, inspiration and information. It would not be possible for us to complete the same without his sincere and affectionate help. His guidance and advice carried me through all the stages of writing my project.

Date :

1. Sikhakolli Phanindra (19BCI7012)
2. Desaboina Deekshitha(19BCI7068)
3. Mamidala Taran (19BCE7346)

ABSTRACT

This Project report presents and provides an overview of the methods or classifiers used for sentiment analysis on online product data. The purpose of this report is to investigate and compare different methods like Random Forest, Naïve bayes approach, SVM, logistic regression and ANN for sentiment analysis, to find out the accuracy that could be obtained by using different methods on a similar dataset with Amazon customer reviews. The practical work is done using the python language and dataset collected using web Scraping. Sentiment Analysis is the task of analysing all this data, retrieving opinions about these products and services and classifying them as positive or negative or neutral, in other words good or bad or neutral. The key parts of any review of any product are the numeric rating and the textual description provided along with this product. We applied with machine learning algorithms and one deep learning algorithms to classify.

TABLE OF CONTENTS

S. No	Chapter	Title	Page
1.		ACKNOWLEDGEMENT	3
2.		ABSTRACT	4
3.	1	INTRODUCTION	5
	1.1	OBJECTIVE	5
	1.2	OVERVIEW OF MACHINE LEARNING	6
4.	2	LITERATURE REVIEW	7
5.	3	PROPOSED METHODOLOGY	9
	3.1	DATA COLLECTION	10
	3.2	DATA EXPLORATION AND EDA	11
	3.3	DATA PREPROCESSING	11
	3.4	FEATURE ENGINEERING	11
	3.5	DATA IMPUTATION	11
	3.6	DATA SPLITTING	12
	3.7	DATA MODELLING	12
	1	LOGISTIC REGRESSION	12
	2	RANDOM FOREST CLASSIFIER	13
	3	NAÏVE BAYES CLASSIFIER	13
	4	SUPPORT VECTOR MACHINE	14
	5	ANN CLASSIFIER	14
	3.8	DATA EVALUATION	15
6.	4	RESULTS AND DISCUSSION	17
7.	5	CODE	23
8.	6	FUTURE SCOPE	39
9.	7	CONCLUSION	40
10.	8	REFERENCES	41

CHAPTER 1

INTRODUCTION

We have accelerated the transition to a more digital world since the pandemic. Nowadays, buying products online is very simple and convenient. Around 2 billion people worldwide use e-commerce sites to purchase daily necessities and other items online. Online ecommerce sites account for nearly 67 percent over all shopping. The demand for online shopping is rapidly increasing, and the market is growing by 25% to 30% annually. Amazon, Flipkart is a well-known E-commerce website that is used by people all over the world. The purpose of this project is to identify the positive, neutral and negative feedbacks of customers on various products and to build a supervised learning model to polarise large amounts of reviews. Any online item with a large number of positive reviews provides a powerful comment of the validity of the product the other hand, smart gadgets, or any other online item, without reviews puts potential prospects in a state of distrust. Simply put, more reviews appear more accurate. People value other people's opinions and experiences, and reviewing a product is the only way to understand what others think of it. Opinions gathered from users' experiences with specific products or topics directly affect future customer purchase decisions. Similarly, negative reviews frequently cause a loss of sales.

1.1 Objectives

The following are the objectives of this project:

- The primary objective is to benefit customers and assist them in choosing the right product while purchasing.
- This project would carry out sentiment analysis of online product purchasing based on reviews and predict whether this review is 'Positive', 'Neutral', 'Negative'.
- Sentiment analysis is a text method to analyse that finds polarity within text, whether an entire document, paragraph, sentence, or clause, to produce the desired outcome.

- This would also assist the firm in understanding their product from the client's point of view and enhancing the product to its full potential.

1.2 Overview of Machine Learning

Machine learning is a programming model in which computer programmers learn by just doing. Though classified intelligence, artificial intelligence is a misnomer and is recognized as a subset of man-made awareness. To solve a particular problem, an ML model is given relevant information, a process known as "preparing information," in order to extract insight into information that would be unknown to man on a quick scan. Machine learning applications cover a wide range of applications. Consider email filtering and computer vision, both of which are time-consuming tasks to complete on a case-by-case basis. AI powered with decision-making abilities developed through experience and data exposure can solve these types of problems at incredible speed.

Machine learning improvements usually revolve around creating techniques and algorithms, study hypothesized a solution to a problem, and implementing the code into the application as required. There are three types of machine learning algorithms.

In general, supervised learning is described as providing the machine with both inputs and outputs in order for the machine to learn the rule for comparing labeled data with outputs by identifying the function that matches with both input and output. Unsupervised learning is defined as the unlabeled data, with machine learning to detect patterns in the given structured input and generating correct output.

CHAPTER 2

LITERATURE REVIEW

Title	Citation	Author	Methodology	Summary
Sentiment Analysis based Method for Amazon Product Reviews	Jyoti Budhwar, Sukhdip Singh, 2021, Sentiment Analysis based Method for Amazon roduct Reviews, INTERNATIONAL JOURNAL OF ENGINEERING RESEARCH & TECHNOLOGY (IJERT) ICACT – 2021 (Volume 09 – Issue 08),	Jyoti Budhwar, Sukhdip Singh	The study utilized a hybrid approach that combined both Nave Bayes, KNN, and LSTM approaches. The naïve bayes algorithm identifies the dataset, whereas the KNN algorithm assists in grouping. In order to enhance accuracy, the data set was trained to use an LSTM-based model. For doing sentiment analysis, a data set of customer reviews would be evaluated.	To summaries, the presented study utilized Naive Bayes, SVM, KNN, and LSTM. The goal is research is to discover more accessible and accurate products. The result for classification was provided by Nave Bayes. KNN also assists in grouping. To enhance solution accuracy, the data set would be trained to use an LSTM-based model. The proposed research is designed to address an identified problems in previous research during sentiment analysis.
Comparative study of machine learning approaches for amazon reviews	A. S. Rathor, A. Agarwal, and P. Dimri, “Comparativ e study of machine learning approaches for amazon reviews, ” Procedia computer science, vol. 132, pp. 1552–1561, 2018.	A.S.Rathor, A. Agarwal, and P.Dimri	SVM, the proposed model for the machine learning approach, may be regarded the baseline learning method for Amazon evaluations because to its great precision. We may conclude that "more precise the data, more precise the outcomes can be obtained" by giving priority to the reviews. SVM accuracy is 81.20%, NB accuracy is 77.42%, while ME accuracy is 70.35.	According to the results, customer reviews are extremely important in the decision-making process. The most surprising and exciting part is classifying opinions as positive, negative, or neutral. The accuracy of classifiers is identical after comparing three algorithms for machine learning employing unigrams features and weighted unigram features in Amazon reviews.

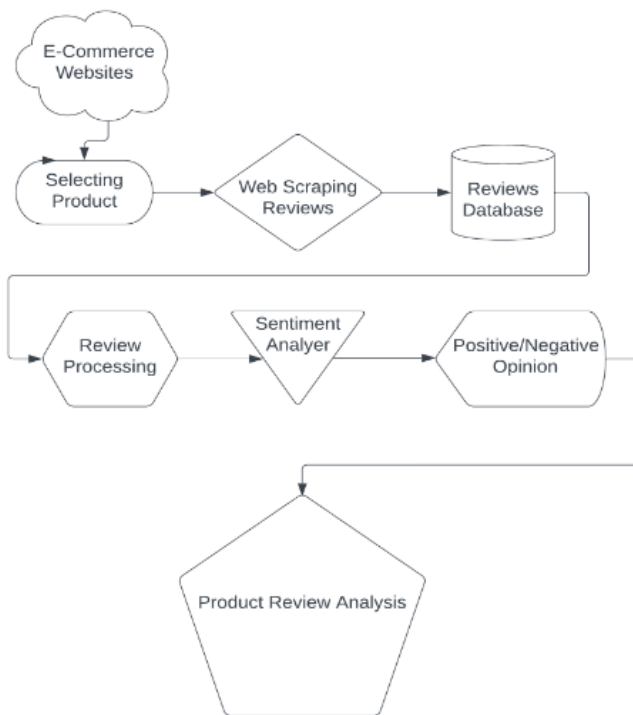
Scientific Text Sentiment Analysis using Machine Learning Techniques	International Journal of Advanced Computer Science and Applications(IJACSA), 10(12), 2019. http://dx.doi.org/10.14569/IJACSA.2019.0101222	Hassan Raza, M. Faizan, Ahsan Hamza, Ahmed Mushtaq and Naeem Akhtar	It assists classifiers achieve the highest accuracy results. They developed a system which utilizes six different machine learning algorithms, including Nave-Bayes (NB), SVM, Logistic Regression (LR), Decision Tree (DT), K-Nearest Neighbor (KNN), and Random Forest (RF).	This method shown significant enhancement, for example, in the scenario of Nave Bayes, they achieved micro-F 87% while the base system described the result as micro-F = 78% and they got the results that really are approximately 9% better. This show that SVM performs better than other classifiers. After SVM, Nave Bayes performs well.
Sentimental Analysis on Amazon Reviews Using Machine Learning	Patil, R.C., Chandrashekar, N.S. (2022). Sentimental Analysis on Amazon Reviews Using Machine Learning. In: Karuppusamy, P., García Márquez, F.P., Nguyen, T.N. (eds) Ubiquitous Intelligent Systems. ICUIS 2021. Smart Innovation, Systems and Technologies, vol 302. Springer, Singapore. https://doi.org/10.1007/978-981-19-2541-2_37	<u>Rajashekhar gouda C. Patil & N. S. Chandrashekar</u>	Random forest, the proposed model for the Machine Learning Technique, can be regarded as the main method for Amazon reviews due to its greatest accuracy, so we can conclude that "more precise the data, the more exacts results can be generated accuracy with 89	This paper conducted a sentimental analysis of Amazon Alexa reviews and developed a model to predict the sentiment of a comment depending on the review text. The project's disadvantage is the sentiment analysis prediction accuracy is still just about 80%.
A Comparative Study of Support Vector Machine and Naive Bayes Classifier for Sentiment Analysis on	2020 International Conference on Contemporary Computing and Applications (IC3A) Available: https://ieeexplore.ieee.org/document/8376299	Sanjay Dey, Sarhan, Dhiman Sikder , SubrinaSultana1, Jayjeet Sarkar, and Monisha Dey	After the preprocessing procedure, the models were trained using nearly 2250 features and nearly 6000 datasets. In the meantime, nearly 4000 test sets have been performed through the statistical measurement models. In this research, a comparison of two machine learning	Based on statistical measures, the goal of this study is to find comparatively better machine learning algorithms among SVM and Naive Bayes classifier. The SVM classifier inside the system has an accuracy of 82.853%, a recall of 82.884%, and a f1 score of 82.662%. The experimental results showed that the support vector machine can polarise

Amazon Product Reviews			methods for analysing the sentiment of customer reviews on Amazon products is presented.	Amazon product feedback with more accuracy.
------------------------	--	--	--	---

CHAPTER 3

PROPOSED METHODOLOGY

We have to get information from the required data source in order to run sentiment analysis (here amazon) This data goes through a number of pre-processing procedures that make it more machine-sensible to perform the task. Our methodology is depicted. First of all, we used scraped dataset which was collected using Web scraping. We used python-based machine learning library named Scikit-Learn for implementing the system. Scikit-Learn is a well-known machine learning library tightly integrated with Python language and provides easy-to-interact interface. First of all, our system reads the data stored after extracting. After reading, pre-processing phase.



Figure

no. 1

Data Pre-processing is applied to clean and prepare the data for the use of machine learning algorithms. Directly text data cannot be given to machine learning algorithms, it should be converted into a suitable type. Using Scikit-Learn module named “count vectorizer”, the text data firstly convert into numeric format and prepare the matrix of tokens count, now the data is ready for machine learning algorithms. Then 80% of data is splatted randomly to train the classifier and 20% for testing the classifier’ accuracy.

3.1 Data Collection

Collecting a large amount of data about any subject from various web pages and articles that must be stored in an appropriate way. Web scraping may be useful in many scenarios when you require to extract a large amount of data quickly. We used Python and BeautifulSoup to extract product reviews from different categories.

Web scraping, also known as data mining, is the process of collecting large amounts of information from the internet and saving it in databases for future analysis and use. Web scraping gives data on price data, market dynamics, latest situation, your competition’ methods, and the challenges they face. If you know how to get it, this data is readily available. Many marketers are unaware of how helpful it could be to them. In this article, we will discuss the advantages of Web scraping and provide a practical example that could be useful for e-commerce businesses and online marketers.

To start with, the purpose of website scraping is to collect web data in HTML or Json format. Python contains the Requests library, which is a simple HTML library that allows you to connect Python code with internet platforms. You could obtain the necessary data for your project from the webpage once you’ve found it. Python provides another library called BeautifulSoup that enables you to grab specific content from a web page, remove HTML elements, and save the data.

The final step in web scraping is to save the data in an structured format. You may save data in the preferred format using the Python Pandas Library. We selected the Amazon website for data collection because Amazon is one of the largest online business enterprises, selling a lot of various kinds of goods all over the world through the Amazon.com platform. Using multiple searches, for examples (latest mobiles, latest laptops other product searches) using

many star ratings and reviews with various products. Fetch information about the item, including its name, price, reviews, and rating, and save it to a CSV file using pandas.

```
In [1]: import pandas as pd
import numpy as np
import re

**Reading the Dataset

In [2]: dataset = pd.read_csv("latest_samsung_mobiles_reviews.csv")
```

Figure no.2

3.2 Data Exploration and EDA

EDA is the evaluative process of data execution to identify designs, solve issues, and test hypotheses using analytical and visual representations. EDA is performed out by using sample data sets. By performing EDA on a sample data set, we can acquire more and more perceptions. We applied various analytical processes to the filtered drug review data set in this work.

3.3 Data Pre processing

We deleted the useless review data during the pre-processing stage. Stopword Removal and REGEXP were used to pre-process reviews by removing undesired terms such as symbols, special characters, and non-English language words.

The following are some of the steps involved in text pre-processing:

- Stopword elimination
- Word mapping from non-standard (slang) to standard
- Possession tagging
- Positive/negative word tagging
- Stemming

3.4 Feature Engineering

Feature Selection

Machine learning algorithms could only work with fixed-length vector of numbers, never raw text, so the input (in this case, text data) should be processed. The Bag of words model of text, which is a commonly used method of feature extraction, is used to transform the texts into features. The technique works by creating individual bags of words from the training data set, with each word assigned a unique number. Using NLP techniques such as BOW or TF-IDF, convert textual data into vector form.

3.5 Data Imputation

The below figure shows the sum of missing values that each column has

```
In [8]: dataset.isnull().sum()

Out[8]: titles      6
        reviews   56
        ratings    6
        dtype: int64
```

Figure

no. 3

After encoding for few columns there were no missing values.

3.6 Data Splitting

We used 80% of our data into training dataset and 20% of our data into testing dataset

```
In [28]: from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(data2['Review'],data2['emotion'],test_size=0.2,random_state=0)
```

Figure

no. 4

3.7 Data Modelling

Here trained the data with various machine learning algorithms. Each classifier algorithm must be trained before being tested in order to successfully carry out the tests. The data was divided into two data sets, training and testing, in order to train and use the classifiers.

1. Logistic Regression

The logistic regression classification approach is used to resolve the binary classification problem. In models with a dual situation, the outcome is usually presented as 0 or 1. Logistic regression is a classification detecting model. Logistic regression predicts the posterior probability of $P(y/x)$ directly by learning the input to output mapping while minimising the error. The prediction is derived by applying binary classification with logistic regression to the data set's training and test data. Naive Bayes

is used for in-text classification because it makes the assumption of independence and performs well in multi-class situations. It exceeds other algorithms in terms of success rate due to its speed and efficiency.

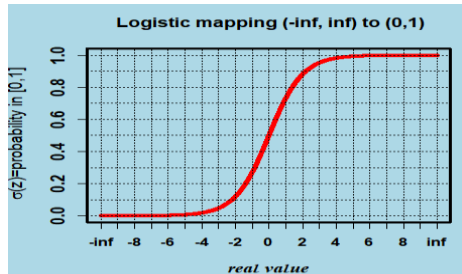
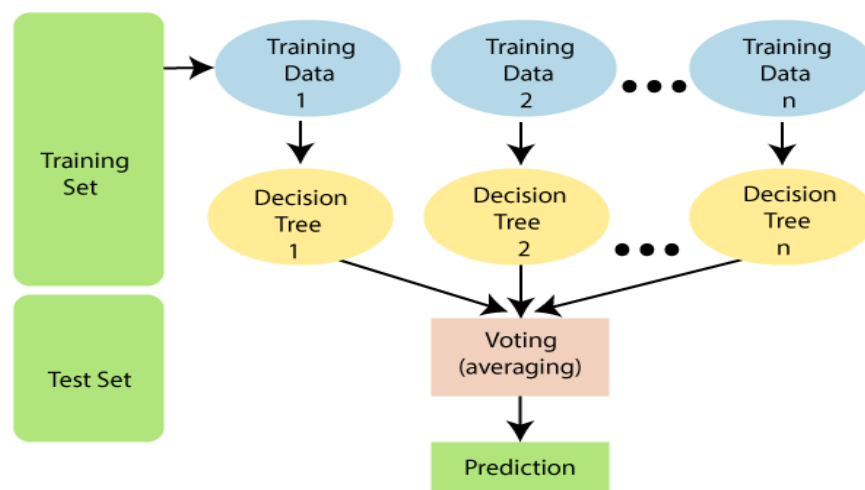


Figure no.5

2. Random Forest Classifier

The random forest approach may be utilized to both classification and regression problems. Ensemble classification methods, which are learning algorithms that construct multiple classifiers rather than just one, can be utilized to classify new data points. The three most frequently utilized ensemble classifiers are bagging, boosting, and random forest. The Random Forest Algorithm eliminates overfitting since the outcome is based on the average or majority vote. The isolation of each decision tree created shows the parallelization property. One of the Random Forest's best features is its ability to handle missing values, providing it a good alternative for anyone searching for a quick and effective method for building a model



Source: google

Figure no.6

3. Naïve Bayes Classifier

The Bayes theorem supports this algorithm. This is one of the quickest and most effective classification algorithms, providing rapid predictions. It generates predictions based on an object's probability.

The Nave Bayes Model is classified into three types:

- Bernoulli
- Gaussian
- Multinomial

The formula for Bayes theorem is

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

4. Support Vector Machine (SVM)

The SVM algorithm's objective is to determine the optimal line or decision boundary for classifying n-dimensional space so that we can easily place new datasets in the right category in the future. A hyperplane is the optimal choice boundaries identifies the extreme points/vectors that assist in the construction of the hyperplane. These extreme examples are known to as support vectors, and the algorithm is recognized as the Support Vector Machine. Consider the diagram below, which illustrates 2 separate categories that are divided by a decision boundary or hyperplane.

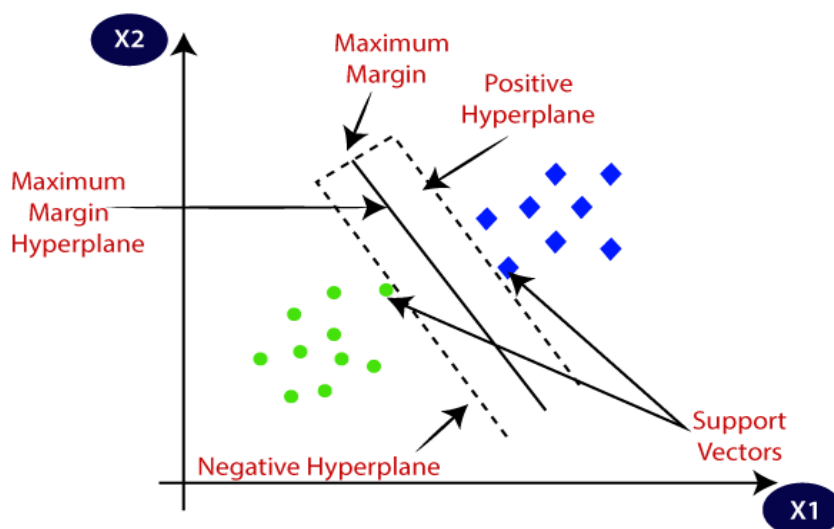


Figure no. 07

5. ANN Classifier (Artificial neural network)

An artificial neural network is a biologically inspired computational model comprised of thousands of single units, called artificial neurons, which are connected together through coefficients (weights) that create the neural structure. Since they process information, they are also known as processing elements (PE). Each PE has a transfer function, weighted inputs, and one output. PE is simply an mathematical equation which combines inputs and outputs. Because the connection weights represent the system's memory, ANNs are also called as connectionist models. Although a single neuron can perform certain basic information processing functions, the power of neural computations was obtained from connecting neurons in a network. The purported intelligence of artificial neural networks is disputed. Artificial neural networks typically have a few hundred or thousand PEs, but the human brain has 100 billion neurons. Artificial networks the with complexity of a human brain are thus far beyond the human brain's creative capacity. The human brain is much more complex, and many of its intellectual functions are still understood. ANNs, on the other hand hand, are capable of processing massive volumes of data and generating predictions that are sometimes surprisingly accurate. Because this doesn't make them intelligent in the traditional 'human' sense, the term computer intelligence may be a better way to characterize these systems. There are many various types of neural networks that have designed, and new ones are developed every week, but these can all be described by the transfer functions of their neurons, the learning rule, and the connection formula.

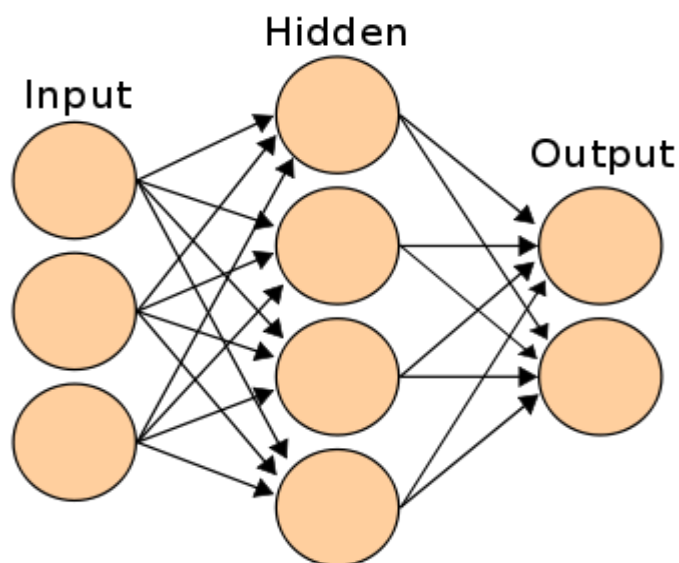


Figure no. 08

3.8 Data Evaluation (Evaluation Metrics)

Evaluating Metrics: Evaluating metrics are important for evaluating classification performance. The most common measure for this purpose is accuracy. A classifier's accuracy on a given test dataset is the percentage of those datasets correctly classified by the classifier. And, because the accuracy measure has never been sufficient to draw a positive decision in the text mining technique, we also used some other metrics to evaluate classifier performance. Precision, recall, and F-measure are three important measures that are frequently used.

		Actual	
		Positive	Negative
Predicted	Positive	True Positive	False Positive
	Negative	False Negative	True Negative

Figure no. 09

Before diving in various measures, there are some terms we need get familiar with: TP (True Positive) represents a number of information points that were successfully classified. The number of accurate data misclassified is indicated by FP (False Positive). The number of incorrect data identified as correct is indicated by FN (False Negative). The amount of inaccurate data classed as TN (True Negative).

CHAPTER 4

RESULTS AND DISCUSSIONS

By successfully implementing on our data, we got an accuracy of about 95.0%. Then we created confusion matrix to check with the counts.

Machine Learning

****Logistic Regression - Accuracy - 91%**

```
In [33]: from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
tvec=TfidfVectorizer()
clf = LogisticRegression(solver='lbfgs')
classifier=Pipeline([('vectorizer',tvec),('classifier',clf)])
classifier.fit(X_train, y_train)
```

	precision	recall	f1-score	support
-1	0.92	0.84	0.88	157
0	0.89	0.92	0.91	223
1	0.93	0.95	0.94	220
accuracy			0.91	600
macro avg	0.91	0.91	0.91	600
weighted avg	0.91	0.91	0.91	600

Confusion Matrix :
[[132 19 6]
[8 206 9]
[3 7 210]]

Accuracy: 91.33333333333333

****SVM - Accuracy - 93%**

```
In [34]: from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import LinearSVC
tvec=TfidfVectorizer()
clf = LinearSVC()
classifier=Pipeline([('reduce_dim',tvec),('classifier',clf)])
classifier.fit(X_train, y_train)
```

	precision	recall	f1-score	support
-1	0.93	0.90	0.92	157
0	0.93	0.93	0.93	223
1	0.94	0.97	0.96	220
accuracy			0.94	600
macro avg	0.93	0.93	0.93	600
weighted avg	0.93	0.94	0.93	600

Confusion Matrix :
[[141 11 5]
[8 207 8]
[2 5 213]]

Accuracy: 93.5

****Naive Bayes - Accuracy - 84%**

```
In [35]: from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
classifier=MultinomialNB()
tvec=TfidfVectorizer()
clf = MultinomialNB()
classifier=Pipeline([('binarizer',tvec),('multinomialnb',clf)])
classifier.fit(X_train, y_train)
```

	precision	recall	f1-score	support
-1	0.91	0.66	0.76	157
0	0.77	0.91	0.83	223
1	0.89	0.90	0.90	220
accuracy			0.84	600
macro avg	0.86	0.82	0.83	600
weighted avg	0.85	0.84	0.84	600

Confusion Matrix :
[[103 43 11]
[8 202 13]
[2 19 199]]

Accuracy: 84.0

****Random Forest - Accuracy - 95%**

```
In [36]: from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.ensemble import RandomForestClassifier
tvec=TfidfVectorizer()
clf= RandomForestClassifier()
classifier=Pipeline([('vectorizer',tvec),('classifier',clf)])
classifier.fit(X_train, y_train)
```

	precision	recall	f1-score	support
-1	0.97	0.89	0.93	157
0	0.94	0.96	0.95	223
1	0.95	0.98	0.96	220
accuracy			0.95	600
macro avg	0.95	0.94	0.95	600
weighted avg	0.95	0.95	0.95	600

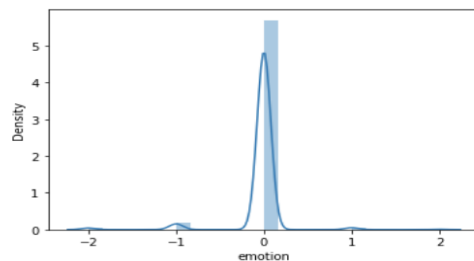
Confusion Matrix :

```
[[140  11   6]
 [   3 214   6]
 [   1   3 216]]
```

Accuracy: 95.0

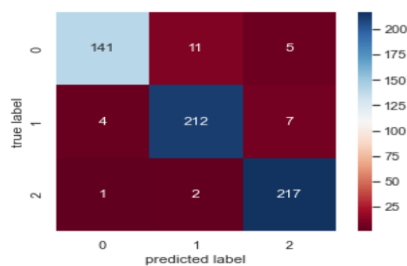
```
In [41]: import matplotlib.pyplot as plt
sns.distplot(y_test-pred)
plt.show()

C:\Users\DELL\anaconda3\lib\site-packages\seaborn\distributions.py:2619: FutureWarning: `d`
will be removed in a future version. Please adapt your code to use either `displot` (a fig-
bility) or `histplot` (an axes-level function for histograms).
warnings.warn(msg, FutureWarning)
```



```
In [42]: %matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns; sns.set()
from sklearn.metrics import confusion_matrix
mat = confusion_matrix(y_test, pred)
axes = sns.heatmap(mat, square=True, annot=True, fmt='d', cbar=True, cmap=plt.cm.RdBu)
plt.ylabel('true label')
plt.xlabel('predicted label')
```

Out[42]: Text(0.5, 12.5, 'predicted label')



```

**Saving the model

In [43]: import pickle
pickle.dump(classifier,open('119.pkl','wb'))
model = pickle.load(open('119.pkl','rb'))

In [44]: pred = classifier.predict(X_test)
pred = pred>0.5

**Testing the model

In [45]: text = ["waste product"]
yp = classifier.predict(text)
print(yp)

[-1]

In [46]: text = ["I Love this Product, quality is good"]
yp = classifier.predict(text)
print(yp)

[1]

In [47]: text = ["It seems 1 size small ... please order 1 size bigger.....so if you are 8 let say order 9.... because this shoes is very
yp = classifier.predict(text)
print(yp)

[-1]

In [48]: text = ["Performance is good charging is fast and enough for average user UI is good no bloatwares Design is ok ok not greaCamera
yp = classifier.predict(text)
print(yp)

[0]

In [49]: text = ['The camera is like the worst ever any phone has released for 20k price range. Even though they say it has 108mp cam the p
yp = classifier.predict(text)
print(yp)

[-1]

```

Deep Learning

```

In [32]: from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test = train_test_split(x,target,test_size = 0.3, random_state = 0)

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Dropout
model = Sequential()

In [33]: model.add(Dense(units = 2000,kernel_initializer = "random_uniform",activation= "relu"))
model.add(Dense(units = 4000,kernel_initializer = "random_uniform",activation= "relu"))
model.add(Dropout(0.2))
model.add(Dense(units = 4000,kernel_initializer = "random_uniform",activation= "relu"))
model.add(Dropout(0.2))
model.add(Dense(units = 1,kernel_initializer = "random_uniform",activation= "sigmoid"))
model.compile(optimizer = "rmsprop",loss = "binary_crossentropy",metrics = ["accuracy"])
model.fit(x_train,y_train, batch_size=32,epochs = 10,validation_data = (x_test,y_test))

Epoch 1/10
66/66 [=====] - 11s 153ms/step - loss: 0.5931 - accuracy: 0.8110 - val_loss: 0.2003 - val_accuracy: 0.
9300
Epoch 2/10
66/66 [=====] - 10s 148ms/step - loss: 0.1278 - accuracy: 0.9567 - val_loss: 0.1260 - val_accuracy: 0.
9578
Epoch 3/10
66/66 [=====] - 10s 146ms/step - loss: 0.0701 - accuracy: 0.9743 - val_loss: 0.1407 - val_accuracy: 0.
9611
Epoch 4/10
66/66 [=====] - 10s 146ms/step - loss: 0.0543 - accuracy: 0.9776 - val_loss: 0.1438 - val_accuracy: 0.
9644
Epoch 5/10
66/66 [=====] - 11s 172ms/step - loss: 0.0349 - accuracy: 0.9829 - val_loss: 0.1600 - val_accuracy: 0.
9622
Epoch 6/10
66/66 [=====] - 13s 199ms/step - loss: 0.0270 - accuracy: 0.9886 - val_loss: 0.1621 - val_accuracy: 0.
9689
Epoch 7/10
66/66 [=====] - 16s 240ms/step - loss: 0.0213 - accuracy: 0.9905 - val_loss: 0.2235 - val_accuracy: 0.
9656
Epoch 8/10
66/66 [=====] - 18s 277ms/step - loss: 0.0223 - accuracy: 0.9881 - val_loss: 0.2032 - val_accuracy: 0.
9633
Epoch 9/10
66/66 [=====] - 17s 258ms/step - loss: 0.0166 - accuracy: 0.9910 - val_loss: 0.2433 - val_accuracy: 0.
9678
Epoch 10/10
66/66 [=====] - 15s 222ms/step - loss: 0.0160 - accuracy: 0.9919 - val_loss: 0.2506 - val_accuracy: 0.
9667

Out[33]: <keras.callbacks.History at 0x214ee810ac0>

```

```

In [39]: text="didn't expect this from one plus. After one month of use the phone started showing some issues mostly in network issue and
text = re.sub('[^a-zA-Z]', ' ',text)
text = text.lower()
text = text.split()

text = [ps.stem(word) for word in text if not word in set(stopwords.words('english'))]
text = ' '.join(text)
yp = model.predict(cv.transform([text]))
yp>0.5

1/1 [=====] - 0s 18ms/step

Out[39]: array([[False]])

In [40]: text="Top notch display, gorgeous display, heavily and seriously powerful device thanks to 8+ gen 1. Tremendous battery life in f
text = re.sub('[^a-zA-Z]', ' ',text)
text = text.lower()
text = text.split()

text = [ps.stem(word) for word in text if not word in set(stopwords.words('english'))]
text = ' '.join(text)
yp = model.predict(cv.transform([text]))
yp>0.5

1/1 [=====] - 0s 18ms/step

Out[40]: array([[ True]])

```

Flask Application

Figure no.10

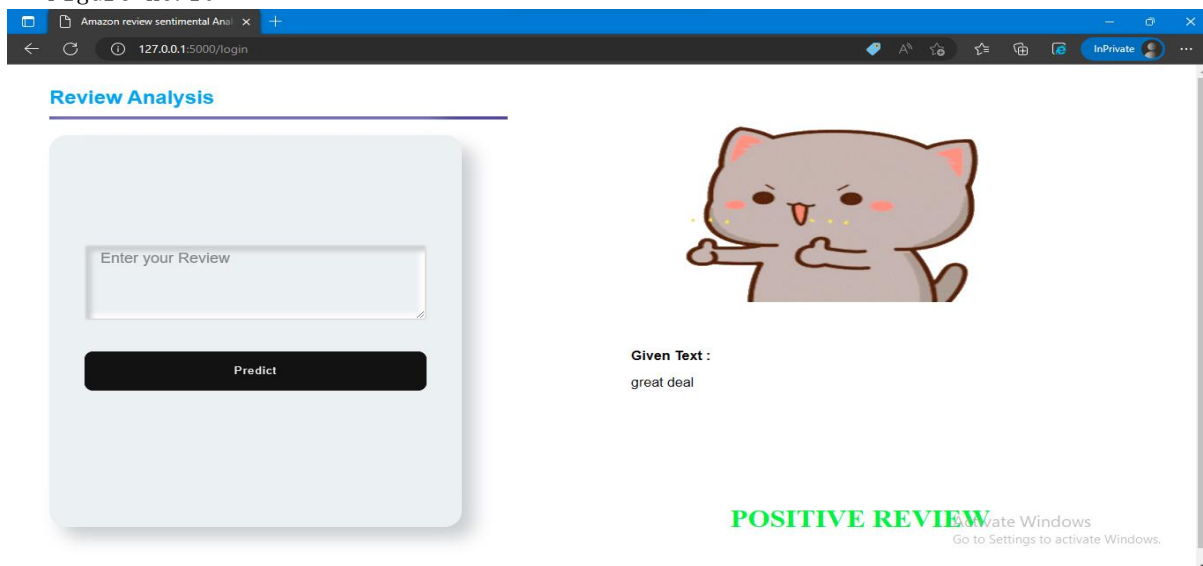


Figure no.11

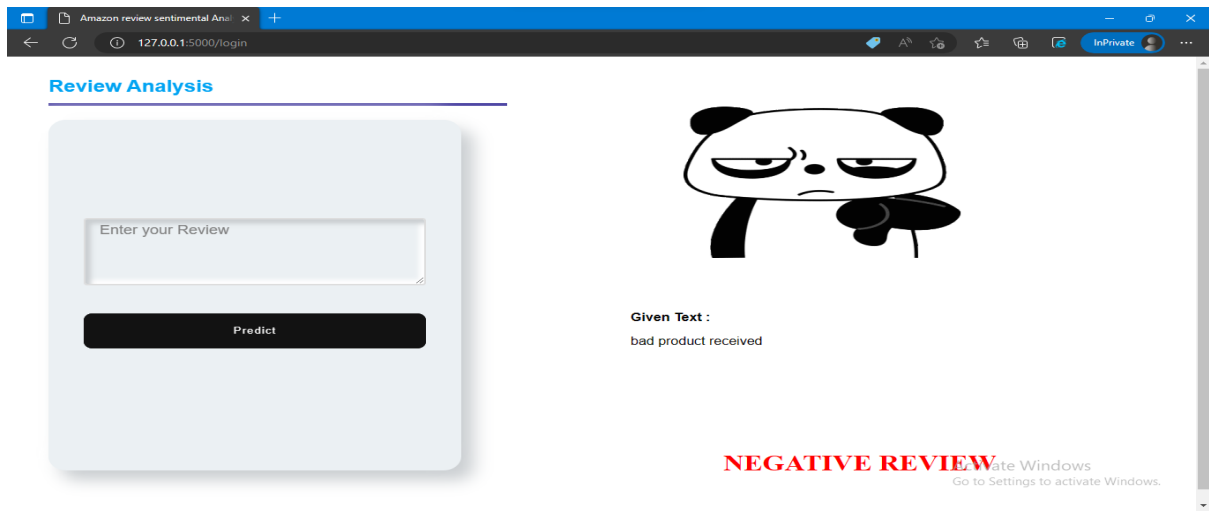
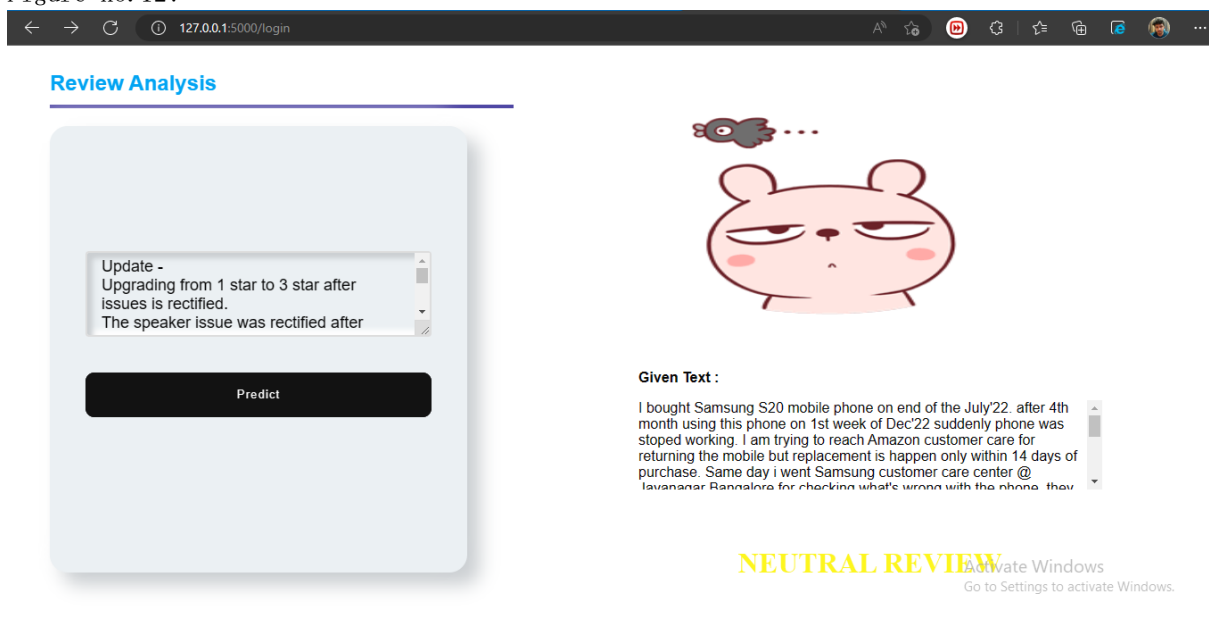


Figure no. 12:



CHAPTER 5

CODE

Web Scrapping

Access the HTML of the webpage and extract useful information/data from it. This technique is called web scraping or web harvesting or web data extraction.

Importing Beautiful Soup Library for Web Scraping

```
In [1]: import requests
        from bs4 import BeautifulSoup

In [3]: search= "best mobiles under 30000" #give product name

In [4]: import re
        search_query = re.sub(r"\\s+", '+', search)
        search_name = re.sub(r"\\s+", '_', search)

In [5]: amazon_base_url="https://www.amazon.in/s?k="

        Get the URL of the webpage you want to scrape.

In [6]: url=amazon_base_url+search_query
        # url="https://www.amazon.in/gp/bestsellers/electronics/1805560031/ref=sq_bs_nav_electronics_3_1389432031"

In [7]: url

Out[7]: 'https://www.amazon.in/s?k=best+mobiles+under+30000'

        Send an HTTP request to the URL of the webpage you want to access. The server responds to the request by returning the HTML content of the webpage.
        For this task, we will use a third-party HTTP library for python-requests.

In [8]: header={'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/77.0.3865.90 Safari/537.36'}

In [9]: search_response=requests.get(url,headers=header)

In [10]: search_response.status_code

Out[10]: 200

In [ ]: search_response.text
```

function to get the soup object from the input search query

```
In [12]: cookies={} # insert request cookies within {}
        def fetchAmazonSearch(search_query):
            url="https://www.amazon.in/s?k="+search_query
            print(url)
            page=requests.get(url,cookies=cookie,headers=header)
            if page.status_code==200:
                return page
            else:
                return "Error"
```

function to get the page of particular product response for that given Asin number

```
In [13]: def Searchasin(asin):
            url="https://www.amazon.in/dp/"+asin
            print(url)
            page=requests.get(url,cookies=cookie,headers=header)
            if page.status_code==200:
                return page
            else:
                return "Error"
```

function to get the review page of particular review Link

```
In [14]: def Searchreviews(review_link):
            url="https://www.amazon.in/"+review_link
            print(url)
            page=requests.get(url,cookies=cookie,headers=header)
            if page.status_code==200:
                return page
            else:
                return "Error"
```

```
In [15]: product_names=[]
        temp = search_query+'&page='
        for k in range(15):
            response=fetchAmazonSearch(temp+str(k))
            soup=BeautifulSoup(response.content)
            for i in soup.findAll("span",{'class':'a-size-medium a-color-base a-text-normal'}): # the tag which is common for all the products
                product_names.append(i.text) #adding the product names to the list
```

```
https://www.amazon.in/s?k=best+mobiles+under+30000&page=0
https://www.amazon.in/s?k=best+mobiles+under+30000&page=1
https://www.amazon.in/s?k=best+mobiles+under+30000&page=2
https://www.amazon.in/s?k=best+mobiles+under+30000&page=3
https://www.amazon.in/s?k=best+mobiles+under+30000&page=4
https://www.amazon.in/s?k=best+mobiles+under+30000&page=5
https://www.amazon.in/s?k=best+mobiles+under+30000&page=6
```

function to get the soup object from the input search query

```
In [12]: cookie={} # insert request cookies within()
def fetchAmazonSearch(search_query):
    url="https://www.amazon.in/s?k="+search_query
    print(url)
    page=requests.get(url,cookies=cookie,headers=header)
    if page.status_code==200:
        return page
    else:
        return "Error"
```

function to get the page of particular product response for that given Asin number

```
In [13]: def Searchasin(asin):
    url="https://www.amazon.in/dp/"+asin
    print(url)
    page=requests.get(url,cookies=cookie,headers=header)
    if page.status_code==200:
        return page
    else:
        return "Error"
```

function to get the review page of particular review link

```
In [14]: def Searchreviews(review_link):
    url="https://www.amazon.in/"+review_link
    print(url)
    page=requests.get(url,cookies=cookie,headers=header)
    if page.status_code==200:
        return page
    else:
        return "Error"
```

```
In [15]: product_names=[]
temp = search_query+'&page='
for k in range(15):
    response=fetchAmazonSearch(temp+str(k))
    soup=BeautifulSoup(response.content)
    for i in soup.findAll("span",{'class':'a-size-medium a-color-base a-text-normal'}): # the tag which is common for all the product names
        product_names.append(i.text) #adding the product names to the List

https://www.amazon.in/s?k=best+mobiles+under+30000&page=0
https://www.amazon.in/s?k=best+mobiles+under+30000&page=1
https://www.amazon.in/s?k=best+mobiles+under+30000&page=2
https://www.amazon.in/s?k=best+mobiles+under+30000&page=3
https://www.amazon.in/s?k=best+mobiles+under+30000&page=4
https://www.amazon.in/s?k=best+mobiles+under+30000&page=5
https://www.amazon.in/s?k=best+mobiles+under+30000&page=6
```

creates a list of data asins for given pages of a product search

```
In [18]: data_asin=[]
for k in range(15):
    response=fetchAmazonSearch('samsung+phones&page='+str(k))
    soup=BeautifulSoup(response.content)
    for i in soup.findAll("div",{'class':"s-result-item s-asin sg-col-0-of-12 sg-col-16-of-20 sg-col s-widget-spacing-small sg-col-16-of-20"}):
        data_asin.append(i['data-asin'])

https://www.amazon.in/s?k=samsung+phones&page=0
https://www.amazon.in/s?k=samsung+phones&page=1
https://www.amazon.in/s?k=samsung+phones&page=2
https://www.amazon.in/s?k=samsung+phones&page=3
https://www.amazon.in/s?k=samsung+phones&page=4
https://www.amazon.in/s?k=samsung+phones&page=5
https://www.amazon.in/s?k=samsung+phones&page=6
https://www.amazon.in/s?k=samsung+phones&page=7
https://www.amazon.in/s?k=samsung+phones&page=8
https://www.amazon.in/s?k=samsung+phones&page=9
https://www.amazon.in/s?k=samsung+phones&page=10
https://www.amazon.in/s?k=samsung+phones&page=11
https://www.amazon.in/s?k=samsung+phones&page=12
https://www.amazon.in/s?k=samsung+phones&page=13
https://www.amazon.in/s?k=samsung+phones&page=14
```

```
In [19]: data_asin
```

```
'B004F52B5X',
'B008GF2P28',
'B00CQW3H3Q',
'B0014PKNS5',
'B00VB2MRFS',
'B00VK3G99F',
'B006VDG9QV',
'B00D3T6Z1D',
'B0014MR9L1',
'B00D3T6MGJ',
'B00NGB2TPR',
'B004F2K7N1',
'B00MGC6LHP',
'B00146VWIV',
'B00VB57558',
'B004F1YC3J',
'B00SH8QD3G',
'B004F3B0EK',
'B00VKCRFP1',
```

```
In [20]: len(data_asin)
```

```
Out[20]: 240
```

creates a list of reviews, ratings and title extracted the text data of asins for given pages of a product search

```
In [23]: reviews=[]
ratings=[]
title=[]
starNum = ['one_star','two_star','three_star','four_star','five_star']
# 50 X 10 = 500 reviews => 5X10pages(each 10reviews) => 10,000
for j in range(5):
    # for k in range(len(starNum)): #<= give no.of pages #(10reviews per page) #### 5(star ratings) X no. of pages X 10(reviews/page)
    for z in range(5): ## 1(no.of pages) X 10(reviews per page) X 5(no.of stars) X 100(no.of products) =
        for k in range(len(starNum)):
            response=Searchreviews(link[j]+'&pageNumber='+str(z)+'&filterByStar='+starNum[k])
            soup=BeautifulSoup(response.content)
            for i in soup.findAll("span",{'data-hook':"review-body"}):
                reviews.append(i.text)
            for p in soup.findAll("i",{'data-hook':"review-star-rating"}):
                ratings.append(p.find_all("span")[0].contents[0])
            for t in soup.findAll("a",{'data-hook':"review-title"}):
                title.append(t.text)
```

```
rType=all_reviews&pageNumber=2&filterByStar=two_star
https://www.amazon.in/Samsung-Galaxy-Storage-6000mAh-Battery/product-reviews/B084F2TTTS/ref=cm_cr_dp_d_show_all_btm?ie=UTF8&reviewe
rType=all_reviews&pageNumber=2&filterByStar=three_star
https://www.amazon.in/Samsung-Galaxy-Storage-6000mAh-Battery/product-reviews/B084F2TTTS/ref=cm_cr_dp_d_show_all_btm?ie=UTF8&reviewe
rType=all_reviews&pageNumber=2&filterByStar=four_star
https://www.amazon.in/Samsung-Galaxy-Storage-6000mAh-Battery/product-reviews/B084F2TTTS/ref=cm_cr_dp_d_show_all_btm?ie=UTF8&reviewe
rType=all_reviews&pageNumber=2&filterByStar=five_star
https://www.amazon.in/Samsung-Galaxy-Storage-6000mAh-Battery/product-reviews/B084F2TTTS/ref=cm_cr_dp_d_show_all_btm?ie=UTF8&reviewe
rType=all_reviews&pageNumber=3&filterByStar=one_star
https://www.amazon.in/Samsung-Galaxy-Storage-6000mAh-Battery/product-reviews/B084F2TTTS/ref=cm_cr_dp_d_show_all_btm?ie=UTF8&reviewe
rType=all_reviews&pageNumber=3&filterByStar=two_star
https://www.amazon.in/Samsung-Galaxy-Storage-6000mAh-Battery/product-reviews/B084F2TTTS/ref=cm_cr_dp_d_show_all_btm?ie=UTF8&reviewe
rType=all_reviews&pageNumber=3&filterByStar=three_star
https://www.amazon.in/Samsung-Galaxy-Storage-6000mAh-Battery/product-reviews/B084F2TTTS/ref=cm_cr_dp_d_show_all_btm?ie=UTF8&reviewe
rType=all_reviews&pageNumber=3&filterByStar=four_star
https://www.amazon.in/Samsung-Galaxy-Storage-6000mAh-Battery/product-reviews/B084F2TTTS/ref=cm_cr_dp_d_show_all_btm?ie=UTF8&reviewe
rType=all_reviews&pageNumber=3&filterByStar=five_star
https://www.amazon.in/Samsung-Galaxy-Storage-6000mAh-Battery/product-reviews/B084F2TTTS/ref=cm_cr_dp_d_show_all_btm?ie=UTF8&reviewe
rType=all_reviews&pageNumber=4&filterByStar=one_star
```

```
In [24]: len(reviews)
```

```
Out[24]: 1120
```

```
In [25]: reviews[:40]
```

```
Out[25]: ['\nPlease , Don\'t buy this product "samsung galaxy M13" specially from Amazon.This product I have purchased from Amazon, I faced
the problem .....The mobile is being hanged continuously and after inserting the sim the network is not showing.....This proble
m I have discussed with customer care and first of all I wanted to return this product and I wanted to take my refund back but the
thing is what they started a process like to send the technician to delivery address and after that the technician called that and
we told him to visit our address and he forced not to come on delivery address. According to technician he said he will resolved th
e issue on call.....one application we installed that he want for testing purpose after that he sent screen-shot on my personal mob
ile number in that he had selected hardware issue and said that now you can return the product. But the main thing is I am unable t
```

```
In [27]: import pandas as pd
import numpy as np
```

```
In [28]: rev={'titles':title, 'reviews':reviews, 'ratings': ratings}
rev
```

```
Out[28]: {'titles': ["\nWorst service provided by Amazon Don't buy this.\n",
'\nBad build quality of Network and bad camera with best battery\n',
'\nVery Bad Product. Dont buy it.\n',
'\nNot up to the standard\n',
'\nWe are ditched\n',
'\nRecieved defective mobile\n',
'\nDON'T BUY THIS PHONE\n',
'\nNot worth the hype and money - Samsung should look at the design quality of M13\n',
'\nToo slow processor\n',
'\nBad display. Contacts app is very pathetic\n',
'\nSamsung M13 5G Phone is not Good. Every thing is slow .\n',
'\nmediocre laggy device\n',
'\nSlow Phone\n',
'\nGood Bujet Smart Phone But *Very Slow*\n',
'\nHang master\n',
'\nPhoto hanging all time\n',
'\nFeatures are not as expected\n',
'\nBudget phone\n',
'\nThis phone is not good\n',
'\n']
```

Creating a dataframe out of all the required elements of the review page

```
In [29]: review_data=[]
review_data = pd.DataFrame.from_dict(rev, orient='index').transpose()
# review_data = pd.DataFrame.from_dict(rev)
# review_data = pd.DataFrame.from_dict([ (k,pd.Series(v)) for k,v in rev.items() ])
```

Detecting Missing Values

```
In [31]: review_data.replace('title', np.nan, inplace=True)
review_data.replace('reviews', np.nan, inplace=True)
review_data.replace('ratings', np.nan, inplace=True)
```

```
In [32]: review_data.sample(5, replace=True)
```

```
Out[32]:
```

	titles	reviews	ratings
665	\Wastage of money\n	\iPhone display stopped working in 2 days only...	2.0 out of 5 stars
189	\Average phone\n	\n\n\n\n\n The media could ...	4.0 out of 5 stars
167	\Average\n	\nAfter 02 months problem comes specially in t...	2.0 out of 5 stars
1018	\Best phone in that money range\n	\nAll good camera finger print\n	5.0 out of 5 stars
32	\nGood but not excellent under this budget\n	\nIt's a decent mobile under this price but fe...	4.0 out of 5 stars

```
In [33]: review_data.dropna(subset=['titles'], inplace=True)
review_data.dropna(subset=['reviews'], inplace=True)
review_data.dropna(subset=['ratings'], inplace=True)
```

```
In [34]: review_data.shape
```

```
Out[34]: (1120, 3)
```

Data Balancing

```
In [39]: len(reviews)
```

```
Out[39]: 1120
```

```
In [40]: len(title)
```

```
Out[40]: 1120
```

```
In [41]: len(ratings)
```

```
Out[41]: 1120
```

```
In [42]: review=pd.DataFrame(review_data.groupby('ratings').size().sort_values(ascending=False).rename('No of Users').reset_index())
review.head()
```

```
Out[42]:
```

	ratings	No of Users
0	5.0 out of 5 stars	250
1	1.0 out of 5 stars	227
2	4.0 out of 5 stars	221
3	3.0 out of 5 stars	216
4	2.0 out of 5 stars	204

```
In [1]: import pandas as pd
import numpy as np
import re
```

****Reading the Dataset**

```
In [2]: dataset = pd.read_csv("latest_samsung_mobiles_reviews.csv")
```

```
In [3]: dataset.head()
```

```
Out[3]:
```

	titles	reviews	ratings
0	\nWorst service provided by Amazon Don't buy t...	\nPlease , Don't buy this product "samsung gal...	1.0
1	\nNot up to the standard\n	\nordered on 12/10/2022 compared to old Samsun...	1.0
2	\nWe are ditched\n	\nVery poor Quality phone has been sent to us ...	1.0
3	\nDONT BUY THIS PHONE\n	\nBought it after seeing 4 star ratings and go...	1.0
4	\nToo slow processor\n	\nl want this product for normal use.....But t...	1.0

```
In [4]: dataset.tail()
```

```
Out[4]:
```

	titles	reviews	ratings
3001	NaN	NaN	NaN
3002	NaN	\n\n\n\n The media could ...	NaN
3003	NaN	\nPhone hanging or spekar to no audio bar bar...	NaN
3004	NaN	\n\n\n\n The media could ...	NaN
3005	NaN	\nPhone hanging or spekar to no audio bar bar...	NaN

****Analyse the dataset**

```
In [5]: dataset.describe()
```

```
Out[5]:
```

	ratings
count	3000.000000
mean	2.819000
std	1.479967
min	1.000000
25%	1.000000
50%	3.000000
75%	4.000000
max	5.000000

```
In [6]: dataset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3006 entries, 0 to 3005
Data columns (total 3 columns):
#   Column   Non-Null Count  Dtype
---  ---
0    titles   3000 non-null   object
1    reviews  2950 non-null   object
2    ratings  3000 non-null   float64
dtypes: float64(1), object(2)
memory usage: 70.6+ KB
```

```
In [7]: dataset.shape
```

```
Out[7]: (3006, 3)
```

****Handling missing values**

```
In [8]: dataset.isnull().sum()
```

```
Out[8]: titles      6
reviews    56
ratings     6
dtype: int64
```

```
In [9]: dataset['reviews'] = dataset['reviews'].fillna('').apply(str)
dataset
```

```
Out[9]:
```

	titles	reviews	ratings
0	\nWorst service provided by Amazon Don't buy t...	\nPlease , Don't buy this product "samsung gal...	1.0
1	\nNot up to the standard\n	\nordered on 12/10/2022 compared to old Samsun...	1.0
2	\nWe are ditched\n	\nVery poor Quality phone has been sent to us ...	1.0
3	\nDONT BUY THIS PHONE\n	\nBought it after seeing 4 star ratings and go...	1.0
4	\nToo slow processor\n	\nl want this product for normal use.....But t...	1.0
...	...	...	...
3001	NaN		NaN
3002	NaN	\n\n\n\n The media could ...	NaN
3003	NaN	\nPhone hanging or spekar to no audio bar bar...	NaN
3004	NaN	\n\n\n\n The media could ...	NaN
3005	NaN	\nPhone hanging or spekar to no audio bar bar...	NaN

3006 rows x 3 columns

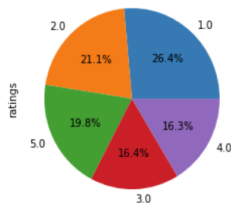
Out[10]:

3000 rows $\times$ 3 columns

```
Out[12]: titles      0
         reviews    0
         ratings     0
         dtype: int64
```

```
Out[13]: 1.0    793
         2.0    633
         5.0    593
         3.0    491
         4.0    490
         Name: ratings, dtype: int64
```

```
Out[14]: <AxesSubplot:ylabel='ratings'>
```



Out[15]: 3000

d

[illegible]

```
In [17]: dt=pd.DataFrame(d,columns=['emotion'])
dt
```

```
Out[17]:
```

	emotion
0	-1
1	-1
2	-1
3	-1
4	-1
...	...
2995	0
2996	-1
2997	-1
2998	-1
2999	-1

3000 rows × 1 columns

```
In [18]: data1=pd.concat([dataset,dt],axis=1)
data1.head()
```

```
Out[18]:
```

	titles	reviews	ratings	emotion
0	\nWorst service provided by Amazon Don't buy t...	\nPlease , Don't buy this product "samsung gal...	1.0	-1
1	\nNot up to the standard\n\nordered on 12/10/2022 compared to old Samsun...		1.0	-1
2	\nWe are ditched\n\nVery poor Quality phone has been sent to us ...		1.0	-1
3	\nDONT BUY THIS PHONE\n\nBought it after seeing 4 star ratings and go...		1.0	-1
4	\nToo slow processor\n\nI want this product for normal use....But t...		1.0	-1

```
In [19]: data1["Review"]=data1[["titles","reviews"]].agg(' '.join,axis=1)
data1.drop(['titles','reviews','ratings'],axis=1,inplace=True)
data2=data1
data2
```

```
Out[19]:
```

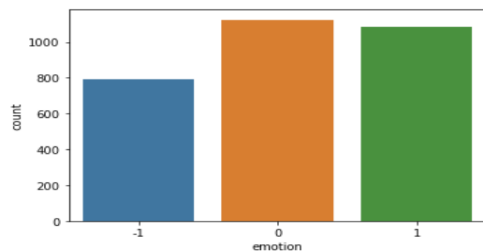
	emotion	Review
0	-1	\nWorst service provided by Amazon Don't buy t...
1	-1	\nNot up to the standard\n\nordered on 12/10/...
2	-1	\nWe are ditched\n\nVery poor Quality phone h...
3	-1	\nDONT BUY THIS PHONE\n\nBought it after see...
4	-1	\nToo slow processor\n\nI want this product f...

****Data Visualization**

```
In [23]: import seaborn as sns
sns.countplot(data2['emotion'])
#sns.countplot(data2['Review'])

C:\Users\DELL\anaconda3\lib\site-packages\seaborn\_decorators.py:36: FutureWarning:
g: x. From version 0.12, the only valid positional argument will be `data`, an
word will result in an error or misinterpretation.
warnings.warn(
```

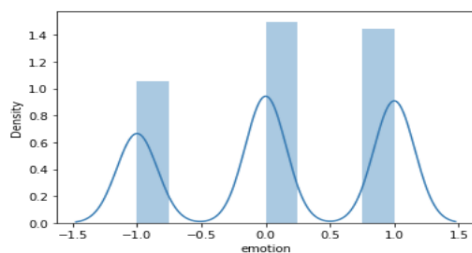
```
Out[23]: <AxesSubplot:xlabel='emotion', ylabel='count'>
```



```
In [24]: sns.distplot(data2["emotion"])
```

```
C:\Users\DELL\anaconda3\lib\site-packages\seaborn\distributions.py:2619: FutureWarning:
will be removed in a future version. Please adapt your code to use either `dis
bility) or `histplot` (an axes-level function for histograms).
warnings.warn(msg, FutureWarning)
```

```
Out[24]: <AxesSubplot:xlabel='emotion', ylabel='Density'>
```



```
In [26]: import nltk
import string
from nltk.corpus import stopwords
stp_words=stopwords.words('english')
from nltk.stem import WordNetLemmatizer
wordnet=WordNetLemmatizer()
from nltk.stem.porter import PorterStemmer
ps = PorterStemmer()
def clean_review(review):
    review=review.lower()
    nopunc=[char for char in review if char not in string.punctuation]
    nopunc=''.join(nopunc)
    cleanreview=" ".join(ps.stem(word) for word in review if word not in stp_words)
    cleanreview=" ".join(wordnet.lemmatize(word) for word in review.split() if word not in stp_words)
    return cleanreview
data2['Review']=data2['Review'].apply(clean_review)
```

```
In [27]: data2.head(10)
```

```
Out[27]:
```

	emotion	Review
0	-1	worst service provided amazon buy this. please...
1	-1	standard ordered 12/10/2022 compared old samsu...
2	-1	ditched poor quality phone sent u . span 10 da...
3	-1	buy phone bought seeing 4 star rating goodl re...
4	-1	slow processor want product normal use....but...
5	-1	worth hype money - samsung look design quality...
6	-1	series phone software . aery range software da...
7	-1	bad display. contact app pathetic display way ...
8	-1	waste money phone. waste money phone. forget a...
9	-1	worst mobile samsung amazon fooled people this...

**Splitting data

```
In [28]: from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(data2['Review'],data2['emotion'],test_size=0.2,random_state=0)
```

```
In [29]: X_train.shape
```

```
Out[29]: (2400,)
```

```
In [30]: X_test.shape
```

```
Out[30]: (600,)
```

```
In [31]: y_train.shape
```

```
Out[31]: (2400,)
```

```
In [32]: y_test.shape
```

```
Out[32]: (600,)
```

Machine Learning

**Logistic Regression - Accuracy - 91%

```
In [33]: from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
tvec=TfidfVectorizer()
clf = LogisticRegression(solver='lbfgs')
classifier=Pipeline([('vectorizer',tvec),('classifier',clf)])
classifier.fit(X_train, y_train)
```

	precision	recall	f1-score	support
-1	0.92	0.84	0.88	157
0	0.89	0.92	0.91	223
1	0.93	0.95	0.94	220
accuracy			0.91	600
macro avg	0.91	0.91	0.91	600
weighted avg	0.91	0.91	0.91	600

Confusion Matrix :

```
[[132 19 6]
 [ 8 206 9]
 [ 3 7 210]]
```

Accuracy: 91.33333333333333

****SVM - Accuracy - 93%**

```
In [34]: from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import LinearSVC
tvec=TfidfVectorizer()
clf = LinearSVC()
classifier=Pipeline([('reduce_dim',tvec),('classifier',clf)])
classifier.fit(X_train, y_train)
```

	precision	recall	f1-score	support
-1	0.93	0.90	0.92	157
0	0.93	0.93	0.93	223
1	0.94	0.97	0.96	220
accuracy			0.94	600
macro avg	0.93	0.93	0.93	600
weighted avg	0.93	0.94	0.93	600

Confusion Matrix :
[[141 11 5]
[8 207 8]
[2 5 213]]

Accuracy: 93.5

****Naive Bayes - Accuracy - 84%**

```
In [35]: from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
classifier=MultinomialNB()
tvec=TfidfVectorizer()
clf = MultinomialNB()
classifier=Pipeline([('binarizer',tvec),('multinomialnb',clf)])
classifier.fit(X_train, y_train)
```

	precision	recall	f1-score	support
-1	0.91	0.66	0.76	157
0	0.77	0.91	0.83	223
1	0.89	0.90	0.90	220
accuracy			0.84	600
macro avg	0.86	0.82	0.83	600
weighted avg	0.85	0.84	0.84	600

Confusion Matrix :
[[103 43 11]
[8 202 13]
[2 19 199]]

Accuracy: 84.0

****Random Forest - Accuracy - 95%**

```
In [36]: from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.ensemble import RandomForestClassifier
tvec=TfidfVectorizer()
clf= RandomForestClassifier()
classifier=Pipeline([('vectorizer',tvec),('classifier',clf)])
classifier.fit(X_train, y_train)
```

	precision	recall	f1-score	support
-1	0.97	0.89	0.93	157
0	0.94	0.96	0.95	223
1	0.95	0.98	0.96	220
accuracy			0.95	600
macro avg	0.95	0.94	0.95	600
weighted avg	0.95	0.95	0.95	600

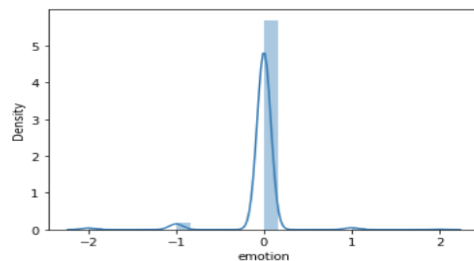
Confusion Matrix :

```
[[140  11   6]
 [   3 214   6]
 [   1   3 216]]
```

Accuracy: 95.0

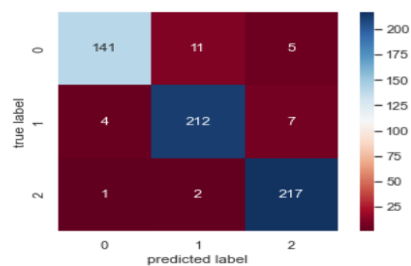
```
In [41]: import matplotlib.pyplot as plt
sns.distplot(y_test-pred)
plt.show()

C:\Users\DELL\anaconda3\lib\site-packages\seaborn\distributions.py:2619: FutureWarning: `d`
will be removed in a future version. Please adapt your code to use either `displot` (a fig
bility) or `histplot` (an axes-level function for histograms).
warnings.warn(msg, FutureWarning)
```



```
In [42]: %matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns; sns.set()
from sklearn.metrics import confusion_matrix
mat = confusion_matrix(y_test, pred)
axes = sns.heatmap(mat, square=True, annot=True, fmt='d', cbar=True, cmap=plt.cm.RdBu)
plt.ylabel('true label')
plt.xlabel('predicted label')
```

Out[42]: Text(0.5, 12.5, 'predicted label')



```

**Saving the model

In [43]: import pickle
pickle.dump(classifier, open('119.pkl', 'wb'))
model = pickle.load(open('119.pkl', 'rb'))

In [44]: pred = classifier.predict(X_test)
pred = pred>0.5

**Testing the model

In [45]: text = ["waste product"]
yp = classifier.predict(text)
print(yp)

[-1]

In [46]: text = ["I Love this Product, quality is good"]
yp = classifier.predict(text)
print(yp)

[1]

In [47]: text = ["It seems 1 size small ... please order 1 size bigger.....so if you are 8 let say order 9.... because this shoes is very
yp = classifier.predict(text)
print(yp)

[-1]

In [48]: text = ["Performance is good charging is fast and enough for average user UI is good no bloatwares Design is ok ok not greaCamera
yp = classifier.predict(text)
print(yp)

[0]

In [49]: text = ['The camera is like the worst ever any phone has released for 20k price range. Even though they say it has 108mp cam the p
yp = classifier.predict(text)
print(yp)

[-1]

```

Deep Learning

```
In [8]: a=[]
        a=dataset["ratings"].tolist()
        len(a)

Out[8]: 3000
```

```
In [9]: d={}
        for i in range(len(a)):
            if a[i]>3:
                d.append(1)
            else:
                d.append(0)
        d

Out[9]: [0,
```

```
[0,
```

```
[0,
```

```
[0,
```

```
[0,
```

```
[0,
```

```
[0,
```

```
[0,
```

```
[0,
```

```
[0,
```

```
[0,
```

```
[0,
```

```
[0,
```

```
[0,
```

```
[0,
```

```
[0,
```

```
[0,
```

```
[0,
```

```
[0,
```

```
[0,
```

```
[0,
```

```
[0,
```

```
In [10]: dt=pd.DataFrame(d,columns=['emotion'])
         dt

Out[10]:
```

	emotion
0	0
1	0
2	0
3	0
4	0
...	...
2995	0
2996	0
2997	0

```
Out[16]: 0      \nWorst service provided by Amazon Don't buy t...
1      \nNot up to the standard\n \nordered on 12/10/...
2      \nWe are ditched\n \nVery poor Quality phone h...
3      \nDON'T BUY THIS PHONE\n \nBought it after see...
4      \nToo slow processor\n \nI want this product f...
        ...
2995   \nCamera quality is very poor\n \nWorst produc...
2996   \nLow quality product and replacement/return i...
2997       \nBad quality m52\n \nBakbass product\n
2998   \nLow quality product and replacement/return i...
2999   \nBad quality m52\n \nDon't buy it. It's not w...
Name: Review, Length: 3000, dtype: object
```

```
Out[17]: 0      0
         1      0
         2      0
         3      0
         4      0
         ..
        2995    0
        2996    0
        2997    0
        2998    0
        2999    0
        Name: emotion, Length: 3000, dtype: int64
```

```
In [19]: data3=[]
for i in range(0,len(input)):
    rev = input[i]
    rev = re.sub('[^a-zA-Z]', ' ',rev)
    rev = rev.lower()
    rev = rev.split()
    rev = [ps.stem(word) for word in rev if not word in set(stopwords.words('english'))]
    rev = [wordnet.lemmatize(word) for word in rev if not word in set(stopwords.words('english'))]
    rev = ' '.join(rev)
    data3.append(rev)
```

```
In [32]: from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test = train_test_split(x,target,test_size = 0.3, random_state = 0)

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.layers import Dropout
model = Sequential()
```

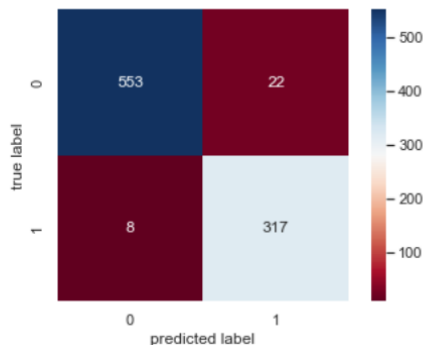
```
In [33]: model.add(Dense(units = 2000,kernel_initializer = "random_uniform",activation= "relu"))
model.add(Dense(units = 4000,kernel_initializer = "random_uniform",activation= "relu"))
model.add(Dropout(0.2))
model.add(Dense(units = 4000,kernel_initializer = "random_uniform",activation= "relu"))
model.add(Dropout(0.2))
model.add(Dense(units = 1,kernel_initializer = "random_uniform",activation= "sigmoid"))
model.compile(optimizer = "rmsprop",loss = "binary_crossentropy",metrics = ["accuracy"])
model.fit(x_train,y_train, batch_size=32,epochs = 10,validation_data = (x_test,y_test))
```

```
Epoch 1/10
66/66 [=====] - 11s 153ms/step - loss: 0.5931 - accuracy: 0.8110 - val_loss: 0.2003 - val_accuracy: 0.9300
Epoch 2/10
66/66 [=====] - 10s 148ms/step - loss: 0.1278 - accuracy: 0.9567 - val_loss: 0.1260 - val_accuracy: 0.9578
Epoch 3/10
66/66 [=====] - 10s 146ms/step - loss: 0.0701 - accuracy: 0.9743 - val_loss: 0.1407 - val_accuracy: 0.9611
Epoch 4/10
66/66 [=====] - 10s 146ms/step - loss: 0.0543 - accuracy: 0.9776 - val_loss: 0.1438 - val_accuracy: 0.9644
Epoch 5/10
66/66 [=====] - 11s 172ms/step - loss: 0.0349 - accuracy: 0.9829 - val_loss: 0.1600 - val_accuracy: 0.9622
Epoch 6/10
66/66 [=====] - 13s 199ms/step - loss: 0.0270 - accuracy: 0.9886 - val_loss: 0.1621 - val_accuracy: 0.9689
Epoch 7/10
66/66 [=====] - 16s 240ms/step - loss: 0.0213 - accuracy: 0.9905 - val_loss: 0.2235 - val_accuracy: 0.9656
Epoch 8/10
66/66 [=====] - 18s 277ms/step - loss: 0.0223 - accuracy: 0.9881 - val_loss: 0.2032 - val_accuracy: 0.9633
Epoch 9/10
66/66 [=====] - 17s 258ms/step - loss: 0.0166 - accuracy: 0.9910 - val_loss: 0.2433 - val_accuracy: 0.9678
Epoch 10/10
66/66 [=====] - 15s 222ms/step - loss: 0.0160 - accuracy: 0.9919 - val_loss: 0.2506 - val_accuracy: 0.9667
```

Out[33]: <keras.callbacks.History at 0x214ee810ac0>

```
In [35]: %matplotlib inline
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns; sns.set()
from sklearn.metrics import confusion_matrix
mat= confusion_matrix(y_test, pred)
axes=sns.heatmap(mat, square=True, annot=True, fmt='d', cbar=True,cmap=plt.cm.RdBu)
plt.ylabel('true label')
plt.xlabel('predicted label')
```

Out[35]: Text(0.5, 12.5, 'predicted label')



```
In [39]: text="didn't expect this from one plus. After one month of use the phone started showing some issues mostly in network issue and
text = re.sub('[^a-zA-Z]', ' ',text)
text = text.lower()
text = text.split()

text = [ps.stem(word) for word in text if not word in set(stopwords.words('english'))]
text = ' '.join(text)
yp = model.predict(cv.transform([text]))
yp>0.5

1/1 [=====] - 0s 18ms/step
```

Out[39]: array([[False]])

```
In [40]: text="Top notch display, gorgeous display, heavily and seriously powerful device thanks to 8+ gen 1. Tremendous battery life in f
text = re.sub('[^a-zA-Z]', ' ',text)
text = text.lower()
text = text.split()

text = [ps.stem(word) for word in text if not word in set(stopwords.words('english'))]
text = ' '.join(text)
yp = model.predict(cv.transform([text]))
yp>0.5

1/1 [=====] - 0s 18ms/step
```

Out[40]: array([[True]])

Flask Application

```
File Edit Selection View Go Run Terminal Help
new_mod.html - cp - Visual Studio Code

templates > new_mod.html > html > style > left
1
2 <!DOCTYPE html>
3 <html lang="en">
4 <script language="JavaScript" type="text/javascript">
5
6     function onlyAlphabets(e, t) {
7         try {
8             if (window.event) {
9                 var charCode = window.event.keyCode;
10            }
11            else if (e) {
12                var charCode = e.which;
13            }
14            else { return true; }
15            if ((charCode > 64 && charCode < 91) || (charCode > 96 && charCode < 123))
16                return true;
17            else
18                return false;
19        }
20        catch (err) {
21            alert(err.Description);
22        }
23    }
24
25 </script>
26 <style>
27     * { box-sizing: border-box; }
28 @import url('https://fonts.googleapis.com/css?family=Rubik:400,500&display=swap');
29
30
31 body {
32     font-family: 'Rubik', sans-serif;
33     /* background-color: #000; */
34 }
35
36 .container {
37     display: flex;
38     height: 100vh;
39 }
40
41 .left {
42     overflow: hidden;
43     display: flex;
44     flex-wrap: wrap;
```

```

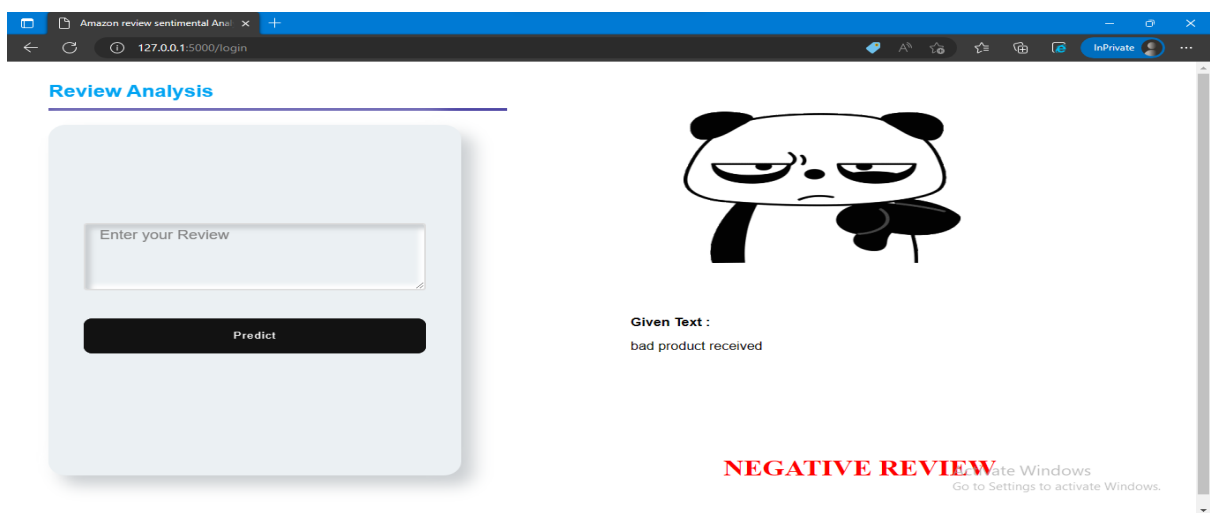
File Edit Selection View Go Run Terminal Help flask_app.py - cp - Visual Studio Code
new_mod.html flask_app.py X
flask_app.py > login
1 import numpy as np
2 import re
3 import nltk
4 from flask import Flask, render_template, request
5 import pickle
6
7 # import pickle
8 import os
9
10 # import TextBlob
11 from textblob import TextBlob
12
13 from nltk.stem.porter import PorterStemmer
14 ps = PorterStemmer()
15
16 PHOTOS_FOLDER = os.path.join('static', 'photos')
17
18 # /static/photos/Positive.gif
19 app = Flask(__name__)
20
21 app.config['UPLOAD_FOLDER'] = PHOTOS_FOLDER
22 model = pickle.load(open('119.pkl', 'rb'))
23
24 @app.route('/')
25 def home():
26     return render_template("new_mod.html")
27
28 @app.route('/login', methods = ['POST'])
29 def login():
30     text = request.form["review"]
31     tempy = TextBlob(text)
32     sample = tempy.correct()
33
34     total = [str(sample)]
35     y_pred = model.predict(total)
36
37     temp = "Given Text : "
38
39     if(y_pred == 1):
40         text3 = os.path.join(app.config['UPLOAD_FOLDER'], 'PosMochi.gif')
41         text2 = "POSITIVE REVIEW"
42         colr = "colgren"
43     elif(y_pred == 0):
44         text3 = os.path.join(app.config['UPLOAD_FOLDER'], 'neutral.gif')

```

```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE
PS D:\capstone_project\cp> & C:/Users/Lenovo/AppData/Local/Programs/Python/Python311/python.exe d:/capstone_project/cp/flask_app.py
d:\capstone_project\cp\flask_app.py:22: DeprecationWarning: Please use 'csr_matrix' from the 'scipy.sparse' namespace, the 'scipy.sparse.csr' namespace is deprecated.
model = pickle.load(open('119.pkl', 'rb'))
* Serving Flask app 'flask_app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
d:\capstone_project\cp\flask_app.py:22: DeprecationWarning: Please use 'csr_matrix' from the 'scipy.sparse' namespace, the 'scipy.sparse.csr' namespace is deprecated.
model = pickle.load(open('119.pkl', 'rb'))
* Debugger is active!
* Debugger PIN: 485-681-399

```



← → ↻ 127.0.0.1:5000/login

Review Analysis

Update -
Upgrading from 1 star to 3 star after
issues is rectified.
The speaker issue was rectified after

Predict



Given Text :

I bought Samsung S20 mobile phone on end of the July'22. after 4th month using this phone on 1st week of Dec'22 suddenly phone was stoped working. I am trying to reach Amazon customer care for returning the mobile but replacement is happen only within 14 days of purchase. Same day i went Samsung customer care center @ lavanagar Bangalore for checking what's wrong with the phone. they

NEUTRAL REVIEW

Activate Windows
Go to Settings to activate Windows.

Amazon review sentimental Anal x +

← ↻ 127.0.0.1:5000/login

Review Analysis

Enter your Review

Predict



Given Text :

great deal

POSITIVE REVIEW

Activate Windows
Go to Settings to activate Windows.

CHAPTER 6

FUTURE SCOPE

Some future works that could be included to improve the model and make it more useful in practical situations. In the future, we intend to employ PCA (Principal Component Analysis) in the active learning process to fully automate data labelling with minimal support from Oracle. The model can be integrated with programmes that communicate with customers looking for a product score. Because we used a large dataset, we can apply the model to local market sites to boost performance and usability. Lastly, we will try to expand this research until we can apply this approach to all kinds of text-based reviews and comments. In this work, we constructed four different types of algorithms with various feature extraction methods. LSTM, KNN, and Maximum entropy are some algorithms that will be used in future studies. Then we will compare the outcome to the outcome of this project.

Our study has certain limitations: NLP Algorithms must be improved; thus, much study is required to understand the area and how it works. Furthermore, we faced several issues with computer memory, which made experiments quite time consuming. We also tried Google Collab to boost performance, but it wasn't enough to run the code.

CHAPTER 7

CONCLUSION

This study investigated the use of various sentiment analysers in combination with machine-learning algorithms to determine the technique with the greatest accuracy rate for learning about election sentiments. In a lexicon-based sentiment analysis, semantic orientation is evaluated for words, phrases, or sentences in a document. Polarity in the lexicon-based technique is evaluated using dictionary, that consists of a semantic score for a specific word. However, the machine learning approach is primarily designed to classify text by applying algorithms such as Nave Bayes and SVM to files. Various techniques were utilised in this study to assess the polarity of the reviews. Nave Bayes, logistic regression, SVM, and Random Forest were the ML techniques used. The Random classifier produced the best results. The Random Forest classifier was 95% accurate, the Logistic regression classifier was 91.3% accurate, the Naive Bayes classifier was 84% accurate, and the SVM classifier was 93.5% accurate and the ANN classifier was 96.6% accurate. This was accomplished through the usage of ANN. Because just a few algorithms were evaluated, it is necessary to test more algorithms or develop hybrid approaches to improve the accuracy of the results.

CHAPTER 8

REFERENCES

- [1] <https://www.analyticsvidhya.com/blog/2021/06/web-scraping-with-python-beautifulsoup-library/>
- [2] <https://ieeexplore.ieee.org/abstract/document/9058128>
- [3] [https://www.researchgate.net/publication/339072438_A_Review_of Online Product Reviews](https://www.researchgate.net/publication/339072438_A_Review_of_Online_Product_Reviews)
- [4] <https://www.kdnuggets.com/2017/01/data-mining-amazon-mobile-phone-reviews-interesting-insights.html>
- [5] <https://ieeexplore.ieee.org/abstract/document/8376299>
- [6] <https://www.kaggle.com/keras/NLP>
- [7] <https://en.wikipedia.org/wiki/Matplotlib#:~:text=Pyplot%20is%20a%20Matplotlib%20module,being%20free%20and%20open%2Dsource.>
- [8] <https://www.aitribune.com/dataset/2018051063>
- [9] <https://deepai.org/dataset/market-1501>
- [10] <https://cocodataset.org/#home>